BASE DE DATOS

2026



Ing. Esteban Schab
AUS. Gabriel Casares
Aux. Alumno Tomas Viñuela

INTRODUCCIÓN SQL

INTRODUCCIÓN



SQL (Structured Query Language): Lenguaje de consulta estructurado.



Toma como base los lenguajes de consultas propuestos por E.F.CODD en 1970 Álgebra Relacional y Cálculo Relacional.



El primer prototipo fue desarrollado por IBM en 1986 para su producto IBM research y SYSTEM R, denominado SEQUEL.



Es el principal motivo del éxito de las bases de datos relacionales.

ESTÁNDAR SQL

- En 1986 se creó el estándar por ANSI (American National Standart Institute) y luego adoptado por ISO (International Standart Organization).
- Se denominó SQL-86 ó SQL-1.
- Fue adoptado por los distintos DBMS.
- En 1992 se crea SQL-2 y posteriormente SQL-3.

ESTÁNDAR SQL

- Permite migrar entre distintos DBMS.
- Si bien los DBMS o SGBD incorporan varias de las definiciones del estándar existen algunas sentencias no soportadas.
- Posee extensiones para conexiones con leguajes de programación como C++, Java, Python, entre otros.

SQL



SQL está dividido en dos grandes partes, ambas ya definidas con anterioridad en esta materia:

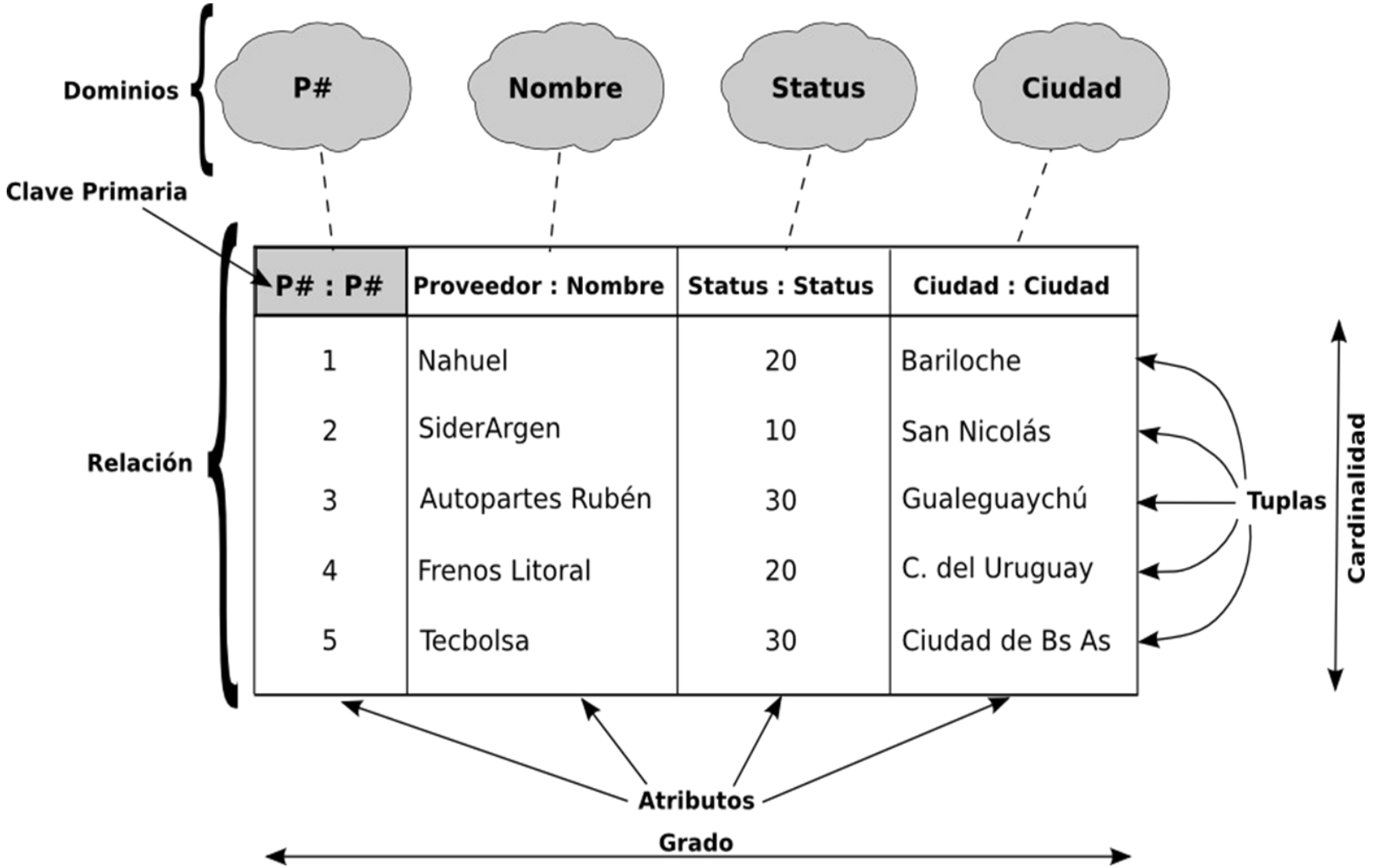


Lenguaje de definición de datos DDL.



Lenguaje de manipulación de datos DML.

SQL


Dominios
P#
Nombre
Status
Ciudad
Clave Primaria
Relación
P# : P#
Proveedor : Nombre
Status : Status
Ciudad : Ciudad
1
Nahuel
20
Bariloche
2
SiderArgen
10
San Nicolás
3
Autopartes Rubén
30
Gualeguaychú
4
Frenos Litoral
20
C. del Uruguay
5
Tecbolsa
30
Ciudad de Bs As
Tuplas
Cardinalidad
Atributos
Grado

SQL

- Si bien se basa en las definiciones formales del modelo relacional, tiene distintas denominaciones, ejemplo:
  - A la relación la denomina table de tabla
  - A los atributos los llama Column o columna

TIPOS DE DATOS

- **Numéricos**
  - Integer, smallint, float, real, decimal(i,j), numeric(i,j)
- **Caracteres**
  - Char, char(n), varchar(n) char varying (n)
- **Bits**
  - Bit(n), bit varying(n)
- **Boolean**
- **Date**
  - Date, time, time with time zone, timestamp

DDL

Sentencias principales:

- **CREATE**
- **DROP**
- **ALTER**

DATABASE

Crear una base de datos

- **CREATE DATABASE *nombredb;***

Ej.: CREATE DATABASE facultad;

Borrar una base de datos

- **DROP DATABASE *nombredb;***

Ej.: DROP DATABASE facultad;

TABLE

CREATE

```
CREATE TABLE alumno(
dni             integer not null,
apellido        varchar(30),
nombres         varchar(40),
tipodni         char not null,
PRIMARY KEY dni,
FOREIGN KEY (tipodni)  REFERENCES tipo(tipodni)
        ON DELETE restrict, ON UPDATE cascade);
```

TABLE

Eliminar

- DROP TABLE alumno;

Modificar

- ALTER TABLE alumno (--)
  - ADD COLUMN mail varchar (50);
  - DROP COLUMN nombre;
  - ALTER COLUMN nombre varchar text;

LENGUAJE DE MANIPULACIÓN DE DATOS - DML

INSERT

- INSERT INTO tabla (lista-de-atributos) VALUES (valores-para-cada-atributo)
- Si se omite la lista-de-atributos se toma como está definida la tabla, en ese orden.

Ej.

INSERT INTO auto (id, marca, modelo, combustible, puertas) VALUES (500, 'AUDI', 'A4','NAFTA',4);

DELETE

- DELETE FROM tabla WHERE (condición)
- Ej. 1

  DELETE FROM alumno WHERE idcarrera=3;
- Ej. 2

  DELETE FROM auto;

  ¿qué hace el ejemplo 2?

UPDATE

UPDATE tabla

SET atributo-que-se-modifica

WHERE condición (opcional)

- Ej.

```
UPDATE producto
SET precio = precio * 1.20
WHERE tipo = 'PERFUMERIA';
```

CONSULTAS, ESTRUCTURA BÁSICA

SELECT *lista de atributos*

FROM lista de *nombres de tablas o subconsultas*

WHERE *condiciones*

* WHERE es opcional.

Ej.

SELECT dni, apellido, nombre

FROM estudiante

WHERE dni > 30000000

RELACIÓN CON A.R.

SELECT *atrib1, atrib2* → PROJECT

FROM *tabla* → nombre de relación

WHERE *predicado* → SELECT de A.R.

Ej.

SELECT dni, apellido

FROM alumno, materia

WHERE alumno.dni = materia.idal

π dni,apellido σ(alumno.dni = materia.idal) (alumno x materia)

SELECT, OPCIONES

SELECT

a, b (lista de atributos)

* (devuelve todos los atributos)

DISTINCT (por defecto SELECT devuelve ALL, todas las tuplas, sin eleminar los duplicados. Si no queremos los duplicados tenemos que indicarle con DISTINCT)

dni AS documento, apell AS apellido (AS se utiliza para renombrar los atributos en la tabla de salida, NO les cambia el nombre en la tabla original).

(sueldo * 1.10) AS aumento

FROM

FROM

- alumno

- alumno a, materia m, carrera c
(se pueden renombrar las tablas, esto sirve, como lo veremos en un ejemplo para hacer más sencilla la redacción de las consultas) En estos casos tanto en el Select como en el Where se utiliza por ejemplo a.dni, m.id, c.nombre, nombre de tabla“.”atributo.

WHERE

WHERE <condiciones>

- sueldo >= 5000 (se pueden utilizar operaciones permitidas para el tipo de dominio de cada atributo).

- alumno.dni = materia.dni AND alumno.legajo = 25054

(se pueden utilizar los operadores AND, OR para filtrar por más de una condición)

BETWEEN

- BETWEEN se utiliza para seleccionar tuplas entre dos valores.
- Va siempre en el WHERE
- WHERE atributo BETWEEN valor1 AND valor2;
- Ej.

```
SELECT apellido, nombre
FROM empleado
WHERE sueldo BETWEEN 10000 AND 15000;
```

LIKE

LIKE se utiliza para comparación de cadenas **%** _

SELECT … FROM …

WHERE **LIKE** nombre *like (*condición usando **%** y/o _)

% reemplaza un substring de cualquier dimensión

_ reemplaza una sóla posición.

LIKE EJEMPLOS

SELECT … FROM …

WHERE **LIKE** nombre *like* 'D%'

todo los nombres que comienzan con D

S… F… WHERE **LIKE** cuit '27_ _ _ _ _ _ _ _ _ _'
todo los cuit que comiencen con 27 y tengan 9 lugares luego.

S… F… WHERE **LIKE** nombre '%cor%'
todos los nombres que contengan 'cor' en el medio.

ORDER BY

ORDER BY le da un orden al conjunto resultado.

Ej:
```
SELECT dni, apellido, nombre, cp
FROM estudiante
WHERE cp = 3260
ORDER BY apellido, nombre DESC;
```

DESC forma descendente

ASC ascendente (por defecto)

EJEMPLOS

- Usuario (dni, genero, apellido, nombre, cp)
- Ciudad (cp, nombre)
- Prestamo (dni, idlibro,fechaprestamo, fechadevolucion)

1) listar los usuarios de Concordia

2) listar las usuarias que hayan retirado algún libro.

3) listar los libros que no fueron devueltos

FUNCIONES DE AGREGACIÓN

- MAX () máximo
- MIN () mínimo
- AVG () promedio
- COUNT () cuenta
- SUM () suma

AGREGACIÓN EJEMPLOS

- SELECT MAX(sueldo), MIN (sueldo) SUM (sueldo), AVG (sueldo)
  FROM empleado e, departamento d
  WHERE e.dpto = d.dpto AND d.nombre = 'SISTEMAS';

- SELECT COUNT (*)
  FROM alumno a, inscripción i
  WHERE a.dni = i.dni AND i.anio = 2016;

FUNCIONES DE GUPO

GROUP BY (agrupa el resultado)

```
SELECT apellido, nombre
  FROM empleado, departamento
  GROUP BY departamento.id
  HAVING (condición de grupo)

SELECT count (*)
  FROM empleado, departamento
  GROUP BY departamento.id
  HAVING sueldo > 15000;
```

¿PREGUNTAS?